🧩 What is Pandas?

Pandas stands for **Python Data Analysis Library**.

It's built on top of **NumPy** and provides **fast, flexible, and expressive data structures** designed to make working with structured (tabular) data easy and intuitive.

Think of Pandas as an Excel sheet inside Python —
but **much more powerful**, **programmable**, and **data-science-friendly**.

🏗️ Core Data Structures in Pandas

Pandas mainly provides **two core classes**:

1. Series → One-dimensional labeled array

Like a **single column** in Excel.

2. DataFrame → Two-dimensional labeled data structure

Like a **full Excel sheet** — rows and columns.

🧠 Step 1: Installing Pandas

pip install pandas

Then import it:

import pandas as pd

🧱 Step 2: Pandas Series (1D Data)

A **Series** is like a Python list with labels (index).

Example 1: Creating a Series

import pandas as pd

data = [10, 20, 30, 40]

s = pd.Series(data)

```
print(s)
```

Output:

```
0    10
1    20
2    30
3    40
dtype: int64
```

Example 2: Custom Index

```
s = pd.Series([10, 20, 30], index=['Math', 'Science', 'English'])
print(s)
```

Output:

```
Math       10
Science    20
English    30
dtype: int64
```

👉 You can access data like a dictionary:

```
print(s['Science'])  # Output: 20
```

📊 Step 3: Pandas DataFrame (2D Data)

A **DataFrame** is like a table with rows and columns.

Example 1: From Dictionary

```
data = {
    'Name': ['John', 'Anna', 'Peter'],
    'Age': [25, 28, 22],
    'City': ['London', 'Paris', 'Berlin']
```

```
}


df = pd.DataFrame(data)

print(df)
```

Output:

```
   Name  Age    City

0   John   25  London

1   Anna   28   Paris

2  Peter   22  Berlin
```

Example 2: Accessing Data

```
print(df['Name'])       # Get a column

print(df.loc[1])        # Get row by label

print(df.iloc[0, 1])    # Get element by position
```

🧮 Step 4: Basic Operations

Let's perform some quick analysis.

```
print(df.head())       # First 5 rows

print(df.info())       # Info about data types

print(df.describe())   # Summary statistics (for numbers)

print(df.shape)        # (rows, columns)
```

🔍 Step 5: Filtering and Selection

Example:

```
# Filter rows where Age > 23

filtered = df[df['Age'] > 23]

print(filtered)
```

Output:

```
  Name  Age   City
0  John   25  London
1  Anna   28   Paris
```

🔁 Step 6: Adding and Removing Columns

```
df['Salary'] = [50000, 60000, 55000]
print(df)


df.drop('City', axis=1, inplace=True)
print(df)
```

📂 Step 7: Reading and Writing Files

From CSV:

```
df = pd.read_csv('data.csv')
```

To CSV:

```
df.to_csv('output.csv', index=False)
```

From Excel:

```
df = pd.read_excel('data.xlsx')
```

📈 Step 8: Simple Analysis Example

Let’s say we have student marks:

```
data = {
    'Name': ['Akhil', 'Kiran', 'Ravi', 'Meena'],
    'Maths': [78, 56, 89, 92],
    'Science': [88, 67, 95, 80]
```

```
}

df = pd.DataFrame(data)

df['Total'] = df['Maths'] + df['Science']
df['Average'] = df['Total'] / 2

print(df)
```

Output:

```
   Name  Maths  Science  Total  Average
0  Akhil     78       88    166     83.0
1  Kiran     56       67    123     61.5
2   Ravi     89       95    184     92.0
3  Meena     92       80    172     86.0
```

📜 Step 9: Summary

Concept	Description	Example
Series	1D labeled data	pd.Series([10, 20, 30])
DataFrame	2D table	pd.DataFrame({'A':[1,2], 'B':[3,4]})
head()	First 5 rows	df.head()
describe()	Summary stats	df.describe()
loc[]	Label-based access	df.loc[0]
iloc[]	Position-based access	df.iloc[1,2]
read_csv()	Load CSV file	pd.read_csv('data.csv')

Concept	Description	Example
to_csv()	Save to CSV	df.to_csv('output.csv')